



BugIt
by Taskivator

Manual completo paso a paso

BugIt

Agente de QA para registrar bugs en VS Code

Configuración inicial y uso diario: una guía detallada, clic a clic, escrita para personas que nunca han hecho algo así antes. No se necesitan conocimientos técnicos. Siga el orden y hoy mismo estará registrando tickets de bug limpios.

Se ejecuta en: VS Code en Windows (principal), macOS & Linux

Aviso sobre la traducción.

Este documento se ha traducido automáticamente y no ha sido revisado por hablantes nativos. La versión en inglés es la que prevalece: en caso de discrepancia, rige el texto en inglés. Para consultar la redacción más precisa y actualizada, acuda al documento en inglés.

CONTENIDO

Haga la **Parte 1** una vez, en orden. Todo lo demás puede consultarlo cuando lo necesite. La **Solución de problemas** y las **Preguntas frecuentes** cerca del final responden las dudas más comunes, así que revíselas antes de escribirnos a soporte.

Parte 1 · Configuración (hágalo una vez)

- Paso 0: Consiga GitHub Copilot
- Paso 1: Instale VS Code
- Paso 2: Descomprima su descarga de Buglt
- Paso 3: Abra Buglt en VS Code
- Paso 4: Elija el agente Buglt en el chat
- Paso 5: Active su licencia
- Paso 6: Acepte los términos (una vez)

Paso 7: Ejecute "Begin setup"

Paso 8: Conecte su sistema de seguimiento

Paso 9: Sus inicios de sesión guardados

Paso 10: Confirme que está listo

Parte 2 · Uso diario

Parte 3 · Hágalo suyo

Parte 4 · Actualizaciones, copias de seguridad y seguridad

Parte 5 · Solución de problemas y preguntas frecuentes

Parte 6 · Licencia y soporte

La única regla que lo mantiene a salvo

Buglt **nunca** registra nada sin antes mostrarle una vista previa y pedirle una confirmación escrita: para las acciones irreversibles debe escribir **FILE IT**, y un simple «sí» no basta. Siempre tiene el control. ¿Quiere practicar sin ningún riesgo? Escriba **dry run** y escribirá el reporte completo **sin** registrarlo: el dry-run suprime todo ticket, comentario, edición, adjunto, eliminación y notificación externos (los comandos locales que usted ejecuta a propósito, como copia de seguridad o exportación, todavía pueden crear archivos locales) e indica al agente que rechace las escrituras en el sistema de seguimiento (ese rechazo sigue las instrucciones de Buglt, no es un bloqueo de la plataforma; use credenciales de solo lectura para evaluar).

Configuración inicial frente a uso diario

Solo sigue toda la **Parte 1** una vez. Después de eso, abrir Buglt es rápido, y las actualizaciones son un solo comando (**Update**). Consulte la Parte 4.

PARTE 1 · CONFIGURACIÓN: HÁGALO UNA VEZ

Unos 15 minutos. Siga los pasos en orden; cada uno se apoya en el anterior. No se salte ninguno.

Consiga GitHub Copilot

No puede usar Buglt sin esto

Buglt es el "conductor" y GitHub Copilot es el "motor" (la IA que realmente escribe los reportes). Si no tiene Copilot funcionando en VS Code, nada de lo que sigue le servirá. Configure esto primero.

Qué es: GitHub Copilot es un asistente de IA de GitHub (propiedad de Microsoft). Tiene un plan gratuito para uso ligero y un plan de pago para uso intensivo. Lo instalará en el Paso 1 e iniciará sesión.

1. Necesitará una **cuenta de GitHub**. Si no tiene una, cree una gratis en github.com/signup.
2. El plan gratuito de Copilot es suficiente para probar Buglt. Si registra bugs todo el día, vale la pena el plan de pago "Copilot Pro", y puede actualizar cuando quiera en github.com/features/copilot.
3. Activará Copilot dentro de VS Code en el Paso 1, así que no necesita hacer nada más aquí todavía.

Instale VS Code

VS Code es una app gratuita de Microsoft. Es la "ventana" en la que vive Buglt.

1. Abra su navegador web y vaya a code.visualstudio.com.
2. Haga clic en el gran botón azul **Download** (detecta su sistema automáticamente).
3. Abra el archivo descargado e instálelo: **accepte todas las opciones predeterminadas**, solo siga haciendo clic en Next/Install.
4. Abra VS Code una vez instalado.

Active GitHub Copilot dentro de VS Code

1. En el extremo izquierdo de VS Code, haga clic en el icono **Extensions** (parece cuatro cuadrados pequeños).
2. En el cuadro de búsqueda, escriba **GitHub Copilot**.
3. Haga clic en **Install** en "GitHub Copilot" (y, si se ofrece, en "GitHub Copilot Chat").
4. Un aviso le pedirá **Sign in to GitHub**. Haga clic en él e inicie sesión con su cuenta de GitHub en la ventana del navegador que se abre.

✓ **Sabrá que funcionó cuando:** aparezca un pequeño icono de Copilot en la parte inferior de VS Code, y se abra el panel de Chat (el icono de burbuja de diálogo a la izquierda, o pulse **Ctrl+Alt+I**).

2

Descomprima su descarga de Buglt

Su compra incluyó un archivo .zip (por ejemplo `buglt-qa-agent.zip`). Necesita descomprimirlo; ejecutarlo comprimido no funcionará.

1. Busque el archivo .zip (normalmente en su carpeta **Downloads**).
2. Haga **clic derecho** en él → elija **Extract All...** (Windows) o haga doble clic en él (Mac).
3. Elija una ubicación sencilla como su carpeta **Documents**, luego haga clic en **Extract**.
4. Ahora tendrá una carpeta **Buglt** con muchos archivos dentro. Recuerde dónde está.

No la ponga en OneDrive ni en una carpeta sincronizada

Las carpetas sincronizadas en la nube pueden causar conflictos de archivos. Su carpeta Documents o el Escritorio son perfectos. Tampoco renombre los archivos de dentro; deje la carpeta tal cual.

3

Abra Buglt en VS Code

1. En VS Code, haga clic en **File** → **Open Folder** (menú superior izquierdo).
2. Vaya hasta donde descomprimió Buglt (p. ej., Documents).
3. Haga clic una vez en la carpeta **Buglt** para seleccionarla, luego haga clic en **Select Folder**.
4. Si VS Code pregunta "Do you trust the authors of the files in this folder?", haga clic en **Yes, I trust the authors**.

✓ **Sabrá que funcionó cuando:** vea los archivos de Buglt (carpetas como `tools`, `.github`, y archivos como `README.md`) listados en el lado izquierdo de VS Code.

4

Elija el agente Buglt en el chat

1. Abra el panel de **Chat** de Copilot: haga clic en el icono de burbuja de diálogo a la izquierda, o pulse `Ctrl+Alt+I` (Windows) / `Cmd+Alt+I` (Mac).
2. En la parte superior del cuadro de chat hay un pequeño menú desplegable de modo (puede decir "Ask" o "Agent").
3. Haga clic en él y elija **Buglt QA Agent** en la lista.

¿No ve "Buglt QA Agent" en la lista?

Asegúrese de haber abierto la carpeta Buglt (Paso 3), no otra carpeta; el agente se carga desde dentro de ella. Cierre y vuelva a abrir la carpeta si es necesario.

5

Active su licencia

Una cuenta de Buglt es todo lo que necesita; no hay ninguna clave de licencia que copiar ni pegar. La activación sucede en su navegador.

1. En el cuadro de chat, escriba `hi` y pulse Enter.
2. Escriba `Activate`. Buglt abre el Buglt Portal en su navegador. Inicie sesión en su propia cuenta, elija su titularidad Solo o Team y apruebe este dispositivo. Vuelva a VS Code y Buglt termina de autorizar automáticamente; su contraseña permanece en el navegador y nunca se introduce en VS Code.

✓ **Sabrá que funcionó cuando:** Buglt confirme la activación y nombre la cuenta con la que ha iniciado sesión.

Mantenga su cuenta en privado

Su cuenta y su titularidad están vinculadas a usted. No las comparta, publique ni revenda; un acceso compartido puede revocarse. Cada miembro del equipo (Team) inicia sesión con su propia cuenta; no hay ninguna clave compartida ni inicio de sesión compartido. Cambiar a otra computadora más adelante no hay problema (consulte las Preguntas frecuentes).

6

Acepte los términos (una vez)

La primera vez, Buglt muestra un breve resumen y le pide que lo acepte antes de hacer cualquier otra cosa.

1. Lea el breve resumen que muestra (usted revisa cada ticket antes de que se registre; la seguridad de su proyecto es su responsabilidad; las salvaguardas son ayudas, no garantías, y como las sigue el modelo de IA que le esté respondiendo en Copilot, un modelo más ligero/gratuito puede a veces saltarse un paso o necesitar que repita un comando, algo que un modelo más potente no haría).
2. Elija **I agree** entre las dos opciones que muestra Buglt.

✓ **Sabrá que funcionó cuando:** Buglt confirme y continúe con la configuración. Solo se le pregunta esto una vez; lo recuerda.

7

Ejecute "Begin setup"

Ahora Buglt se configura solo entrevistándolo en lenguaje sencillo. Nunca edita un archivo de ajustes a mano.

USTED ESCRIBE

Begin setup

Le hará preguntas breves y activará solo lo que elija. La más importante es su sistema de seguimiento.

8

Conecte su sistema de seguimiento

La configuración registró *qué* sistema de seguimiento usa. Este paso es el que permite que BugIt registre tickets en él de verdad: usted crea un token de API en su propia cuenta y lo guarda en esta computadora con un solo comando. BugIt registra los tickets directamente a través de la API REST de su sistema de seguimiento con ese token, así que no hace falta dejar nada en ejecución y el registro nunca usa un servidor MCP.

8a · Cree un token en su propia cuenta

SU SISTEMA DE SEGUIMIENTO	DÓNDE CREARLA
Jira Cloud	id.atlassian.com/manage-profile/security/api-tokens
Azure DevOps	dev.azure.com/YOUR-ORG/_usersSettings/tokens
GitHub Issues	github.com/settings/personal-access-tokens/new
GitLab Issues	gitlab.com/-/user_settings/personal_access_tokens
Bugzilla	en su propio Bugzilla: Preferences → API Keys
YouTrack	en su propio YouTrack: avatar → Profile → Account Security → Authentication
Linear	linear.app/settings/account/security
Shortcut	app.shortcut.com/settings/account/api-tokens
ClickUp	app.clickup.com/settings/apps
Asana	app.asana.com/0/my-apps
Trello	trello.com/apps/admin (una clave de API Y un token que usted autoriza con ella)

Copie su token en cuanto aparezca

La mayoría de los servicios muestran un token nuevo solo una vez. Cópelo de inmediato y guárdelo en un lugar seguro hasta que BugIt lo guarde por usted (Paso 9). Elija la caducidad más larga que ofrezca su sistema de seguimiento para no tener que repetir esto pronto.

8b · Guárdelo en esta computadora

Abra un terminal dentro de VS Code y ejecute el comando de su sistema de seguimiento, usando `ado`, `github`, `gitlab` y demás en lugar de `jira`.

USTED EJECUTA (EN EL TERMINAL)

```
python tools/connect.py jira
```

8c · Pegue el token en el mensaje enmascarado

El comando pide el token en un mensaje local enmascarado: nunca se muestra en pantalla y nunca llega al chat, a un ticket ni a `config.json`. Péguelo ahí y en ningún otro sitio. Buglt inicia sesión, enumera los proyectos que el token ve realmente, lee los tipos de incidencia que usted puede crear y **se niega a guardar una credencial que no pueda registrar tickets**. Por eso, si la configuración termina, el registro funciona.

8d · Compruebe que está conectado

Para confirmarlo en cualquier momento, ejecute la comprobación en el terminal o pregunte en el chat:

USTED EJECUTA (EN EL TERMINAL)

```
python tools/connect.py jira --check
```

USTED ESCRIBE

```
Check status
```

✓ **Sabrá que funcionó cuando:** Buglt indique que su sistema de seguimiento está conectado / en buen estado.

No hace falta dejar nada en ejecución

Su token de API vive en el almacén de credenciales de su sistema operativo, no en VS Code. No hay nada que iniciar cuando vuelva a abrir VS Code: Buglt puede registrar de inmediato. Puede comprobarlo cuando quiera con `python tools/connect.py jira --check`.

9

Sus inicios de sesión guardados

Cuando guarda su token (Paso 8c), sus credenciales quedan guardadas de forma segura en el almacén integrado de su computadora, es decir, el almacén de credenciales de su sistema operativo (Windows Credential Manager / macOS Keychain / Linux Secret Service). Nunca se escriben en un archivo de texto plano y nunca se envían a ningún sitio. No tiene que hacer nada más: la próxima vez que abra VS Code, sus datos guardados ya están ahí.

¿Quiere confirmar qué sistemas de seguimiento ya tienen una credencial guardada? Solo escriba:

USTED ESCRIBE

```
Check saved credentials
```

...y Buglt le muestra qué sistemas de seguimiento tienen una credencial guardada; nunca los valores de las credenciales. Buglt nunca muestra ni copia el valor de un token guardado.

10 Confirme que está listo

Pídale a BugIt que verifique todo dos veces antes de registrar algo de verdad:

USTED ESCRIBE

Check readiness

Enumera todo lo que aún falte (normalmente solo "conecte su sistema de seguimiento" o "active"). Cuando todo esté en verde, verá "✅ Setup complete."

Eso es todo: la configuración está hecha para siempre

No repetirá la Parte 1. A partir de ahora, abrir BugIt es: abrir la carpeta → escribir `hi`. Para cambiar cualquier elección más adelante, solo escriba `Begin setup` y diga qué quiere cambiar.

¿Sin GitHub Copilot? Hay un asistente de terminal

Si no puede usar Copilot, abra la Terminal integrada (Terminal → New Terminal) y ejecute `python tools/setup_wizard.py`. Hace unas pocas preguntas y escribe sus ajustes sin la IA. También puede ejecutar BugIt de forma independiente con su propia clave de IA; consulte las Preguntas frecuentes.

PARTE 2 · USO DIARIO

Una vez configurado, trabaja completamente desde el chat. Hable con naturalidad, o use los ejemplos exactos de abajo. Escriba `help` en cualquier momento para ver la lista completa.

Escribir un reporte de bug

El trabajo principal de BugIt. Describa el bug en una frase sencilla; escribe el ticket completo, autocompleta la versión y la categoría, comprueba si hay duplicados, le muestra una vista previa y lo registra después de que usted escriba `FILE IT` para confirmar (un simple «sí» no basta).

USTED ESCRIBE

Write a bug report: la app se cierra cada vez que toco Guardar en iPhone

Redacta un título, pasos para reproducirlo, resultado esperado frente al real, severidad y plataforma, y siempre ajusta el propio campo de prioridad/severidad del sistema de seguimiento para que coincida, no solo un valor genérico por defecto. ¿Quiere cambios antes de registrarlo? Solo dígalos, por ejemplo "pon la severidad en alta" o "añade que empezó en la última versión."

Bugs rápidos (registro veloz)

Para los casos comunes (crash, error, missing, layout, slow, stuck, data, visual, audio, blocker), el formulario rápido se salta algunas preguntas y completa la categoría y la prioridad por usted, y aun así ejecuta primero la comprobación completa de duplicados.

USTED ESCRIBE (EJEMPLOS)

Quick bug: crash al iniciar tras la actualización

Quick bug: layout el botón se superpone con el texto en la pantalla de ajustes

Quick bug: blocker no puedo continuar después del paso de pago

Quick bug: slow el panel tarda 20 segundos en cargar

Reportes de bugs de fallos (crash)

Si conectó una herramienta de fallos (como Sentry), BugIt hace todo lo anterior además de relacionar su reporte con el fallo y obtener el stack trace por usted.

USTED ESCRIBE

Write a crash bug report: el juego se cierra al abrir el mapa

Comentarios de verificación y cierre

Después de probar una corrección, BugIt escribe un comentario ordenado para publicar en el ticket. Escribe (y opcionalmente publica) el comentario; no cambia el estado del ticket. Usted sigue cerrando/resolviendo/reabriendo el ticket en su sistema de seguimiento.

USTED ESCRIBE	QUÉ OBTIENE
Close #123	Pregunta qué escenario (corrección confirmada, cerrado a petición, funciona como se diseñó, reabrir...) y luego escribe ese comentario.
Escribe un comentario de verificación para 1.2.0 en Windows	Un comentario de "corrección confirmada" con los detalles de la compilación completados.
Escribe un comentario de reapertura para 1.2.0 en Windows	Un comentario de "el problema sigue ocurriendo" (luego usted reabre el ticket).

Traducción

Traduce reportes o términos entre sus idiomas configurados, usando la redacción exacta de su equipo (de su glosario; consulte la Parte 3).

USTED ESCRIBE (EJEMPLOS)

Translate to ja: el botón de guardar no hace nada en la pantalla de perfil

Traduce este reporte de bug al inglés: [pega el texto]

Busque información (especificaciones y glosario)

Si conectó especificaciones o completó su glosario, hágale preguntas a Buglt sobre su proyecto.

USTED ESCRIBE (EJEMPLOS)

¿Qué es [tu término]?

Explicame el flujo de compra

¿Qué hay de nuevo en las especificaciones?

Detección de duplicados

Esto ocurre automáticamente antes de cada registro: Buglt busca en su sistema de seguimiento tickets coincidentes (incluso los redactados de forma un poco distinta) y le avisa, para que nunca registre el mismo bug dos veces.

Funciones avanzadas (si las activó)

ESCRIBA ESTO	QUÉ HACE
<code>Triage digest</code>	Standup instantáneo: bugs abiertos por área/severidad, los más antiguos señalados.
<code>Export bug to file</code>	Guarda el reporte como Markdown/CSV/JSON en lugar de registrarlo (ideal antes de tener el sistema de seguimiento configurado).
<code>Undo last filing</code>	Muestra el último registro del historial de auditoría. Buglt no puede eliminar ni cerrar un ticket; tras una nueva vista previa y <code>FILE IT</code> solo puede eliminar un comentario o un adjunto creados por él mismo.
(automático)	Alertas de SLA en bugs urgentes · campos específicos por categoría.

La lista completa de comandos

USTED ESCRIBE

`help`

...muestra todos los comandos dentro del agente, en cualquier momento.

Practique de forma segura en cualquier momento

Escriba `dry run` y Buglt escribe todo el reporte **sin** registrar nada, perfecto para aprender o probar. El dry-run suprime todo ticket, comentario, edición, adjunto, eliminación y notificación externos (los comandos locales que usted ejecuta a propósito, como copia de seguridad o exportación, todavía pueden crear archivos locales) e indica al agente que rechace las escrituras en el sistema de seguimiento; ese rechazo sigue las instrucciones de Buglt, no es un bloqueo de la plataforma, así que use credenciales de solo lectura si quiere evaluarlo con total garantía.

PARTE 3 · HÁGALO SUYO: AÑADA EL CONOCIMIENTO DE SU PROYECTO

Recién instalado, Buglt es un agente de QA en blanco. Así es como le enseña su proyecto después de la compra. Sin programar, sobre todo hablando con él en el chat. Todo lo que añada permanece en su computadora.

1 · Las palabras de su equipo (el glosario)

Esto hace que las traducciones y la predicción de categorías sean precisas para su proyecto.

1. En la lista de archivos, abra `.github/glossary/terms.template.md`.
2. Complete la tabla con las palabras de su equipo: nombres de funciones, nombres de pantallas, nombres de elementos, abreviaturas y (si usa dos idiomas) sus traducciones.
3. Guarde el archivo (`Ctrl+S`). Eso es todo; Buglt usa estos términos exactos a partir de ahora.

Atajo

Active "glossary auto-seed" durante `Begin setup` y Buglt sugiere términos a partir de sus tickets existentes, y usted solo aprueba cada uno.

2 · Su estilo de bugs (estilo propio)

¿Quiere que los tickets coincidan exactamente con el formato de su equipo: estilo de título, etiquetas de severidad, campos obligatorios? No lo describe; lo **muestra**:

1. En el chat, diga: "que coincida con el estilo de mi equipo."
2. Pegue **una captura de pantalla** de un ticket existente de su sistema de seguimiento.
3. Buglt la estudia (localmente, eliminando antes cualquier dato personal) y guarda su formato, luego lo sigue en cada ticket futuro.

3 · Sus categorías, plataformas y campos

Solo dígame a Buglt en lenguaje sencillo y actualizará sus ajustes por usted:

USTED ESCRIBE (EJEMPLOS)

Mis categorías son Jugabilidad, UI, Audio y Redes
Probamos en Windows y Xbox

4 · Sus propias herramientas (conexiones personalizadas)

¿Usa una herramienta que no está en la lista integrada? Conéctela igual que una integrada:

USTED ESCRIBE

Añade un servidor MCP personalizado

Asígnele un nombre corto y su dirección (debe ser `https://`); Buglt se conecta y comprueba su estado por usted.

5 · Simplemente corrijalo sobre la marcha

La forma más fácil de todas: cuando Buglt redacte un ticket, corrija lo que no le guste: una etiqueta, la redacción, la severidad. Con "learn from your edits" activado (recomendado), recuerda su preferencia y la aplica la próxima vez. El conocimiento de su proyecto se acumula de forma natural, y nada de eso sale nunca de su máquina.

Todo lo que añade es suyo y permanece local

Su glosario, su estilo propio, sus categorías y sus preferencias aprendidas viven en su computadora, se respaldan antes de cada actualización y nunca se envían a nadie.

6 · Comparta su configuración con su equipo (licencia Equipo)

Una vez que haya configurado su glosario, su estilo propio y sus elecciones de sistema de seguimiento como quiere, dele al resto de su equipo exactamente la misma configuración con un solo comando, en lugar de repetir los pasos 1 - 3 en cada máquina:

USTED EJECUTA (UNA VEZ, DESPUÉS DE QUE SU CONFIGURACIÓN ESTÉ COMO QUIERE)

```
python tools/share.py export
```

Eso escribe `config.share.zip` : sus elecciones de sistema de seguimiento/fallos/comunicación, su glosario y su estilo propio, agrupados sin contraseñas ni tokens dentro. Envíeselo a su equipo.

CADA COMPAÑERO EJECUTA

```
python tools/share.py import config.share.zip
```

Obtienen su terminología exacta, su estilo de bugs y su configuración de sistema de seguimiento de inmediato, sin volver a escribir categorías y sin volver a pegar una captura para el estilo propio. Su propia licencia y los ajustes de su dispositivo no se tocan.

Si la importación cambia hacia dónde se envían los datos

Si una configuración compartida añade una nueva conexión personalizada o dirección de notificación, Buglt muestra exactamente qué cambió antes de que ocurra cualquier otra cosa; no se aplica en silencio. También toma primero una instantánea de la configuración actual del compañero, así que [Restore my settings](#) deshace la importación si no era lo que esperaba.

PARTE 4 · ACTUALIZACIONES, COPIAS DE SEGURIDAD Y SEGURIDAD

Obtener actualizaciones (un solo comando)

Cuando hay una versión más reciente disponible, Buglt la menciona una vez cuando dice `hi`. Las actualizaciones de software están incluidas mientras su licencia de Buglt esté activa. Para instalarla:

USTED ESCRIBE

```
Update
```

1. Buglt hace primero una copia de seguridad reciente de sus archivos.
2. Descarga la nueva versión y comprueba que es genuina y no manipulada (está firmada criptográficamente, así que se rechaza una actualización falsa o alterada).
3. La instala sin tocar sus ajustes, su glosario, su estilo propio, su licencia ni sus tokens.
4. Le pide que recargue VS Code (`Ctrl+Shift+P` → escriba "Reload Window" → Enter). Inicie un chat nuevo y estará en la nueva versión.

Nunca se elimina ninguna carpeta

A diferencia de la primera instalación, las actualizaciones son in situ. Nunca elimina ni vuelve a descargar nada, y su configuración siempre se conserva y se respalda.

Qué es seguro editar a mano y qué no

Seguro, y se conserva en cada actualización: `config.json`, `.github/instructions/house-style.instructions.md` (consulte la Parte 3; es la forma admitida de personalizar el comportamiento), `.github/glossary/terms.template.md`, `.github/instructions/learned.instructions.md`, y `.vscode/mcp.json`.

No es seguro: no edite esto a mano

El archivo del agente (`.github/agents/bugit-qa-agent.agent.md`), cualquier otro archivo bajo `.github/instructions/`, y todo lo que haya en `tools/` son el producto en sí, no su configuración. Una actualización los **sobrescribe en silencio**, y, a diferencia de los archivos anteriores, **no hay copia de seguridad desde la que restaurarlos**. Buglt comprueba esto al iniciar y de nuevo justo antes de instalar una actualización, y le avisa si encuentra algún cambio, pero la regla más segura es simplemente no editarlos.

Copias de seguridad y restauración

USTED ESCRIBE

QUÉ HACE

```
Back up my settings
```

Guarda una instantánea local de su config, glosario, estilo propio, endpoints MCP y aprendizaje. Su titularidad de dispositivo firmada y los valores de credenciales se excluyen.

```
List backups
```

Muestra todas las instantáneas guardadas con su fecha.

Restore my settings

Retrocede a una instantánea (y antes toma una instantánea de su estado actual, para que la propia restauración se pueda deshacer).

Automático antes de cada actualización

Buglt siempre hace una copia de seguridad antes de instalar una versión nueva, así que una actualización nunca puede perder su configuración. Si algo se ve mal, [Restore my settings](#) lo soluciona.

Seguridad: qué está protegido

✓ Nada sin su confirmación escrita

Cada ticket y comentario se muestra en vista previa; las acciones irreversibles requieren escribir [FILE IT](#), y un simple «sí» no basta.

🧪 Dry-run = cero escrituras

Diga [dry run](#) y Buglt solo lee. El dry-run suprime todo ticket, comentario, edición, adjunto, eliminación y notificación externos (los comandos locales que usted ejecuta a propósito, como copia de seguridad o exportación, todavía pueden crear archivos locales) e indica al agente que rechace las escrituras en el sistema de seguimiento; ese rechazo sigue las instrucciones de Buglt, no es un bloqueo de la plataforma, así que use credenciales de solo lectura para una evaluación garantizada.

🔍 Sin secretos en archivos

Los tokens viven en el almacén de su sistema operativo, nunca en un archivo de texto plano, nunca se envían a nosotros.

🔒 Se ejecuta en su máquina

Solo habla con las herramientas que conecta y con su propio modelo de IA.

Sus datos y privacidad

Lo único que Buglt envía a Taskivator son datos de licencia/actualización: el inicio de sesión en su cuenta de Buglt (en el navegador, en el Portal), un identificador de dispositivo con hash unidireccional, un identificador de instalación aleatorio, la etiqueta de su dispositivo (nombre de host) y el nombre de su sistema operativo, la versión de Buglt y el filtro de plan que elija, material criptográfico de desafío de vida corta, y la titularidad Solo o Team que aprueba para este dispositivo. No hay ninguna clave de licencia, y su contraseña permanece en el navegador y nunca se introduce en VS Code. Sus reportes de bug, especificaciones, glosario, ajustes y tokens nunca salen de su máquina. Sin telemetría, sin analíticas, nunca. Los detalles completos están en el archivo [PRIVACY.md](#) de su carpeta Buglt.

Riesgos y advertencias importantes: por favor, léalo

La IA puede equivocarse

Buglt redacta reportes con un modelo de IA, que puede malinterpretar detalles o inventar algo. Lea siempre la vista previa antes de confirmar por escrito ([FILE IT](#) para comentar; un simple «sí» no basta). Usted está aprobando lo que se va a registrar.

Registra en su sistema de seguimiento real

Una vez que aprueba, se crea un ticket real. ¿Está practicando? Use `dry run` para escribir el reporte sin registrarlo.

La seguridad de su proyecto es responsabilidad suya

Cómo usa BugIt, y la seguridad de sus proyectos, datos y sistema de seguimientos, son su responsabilidad. Las salvaguardas reducen el riesgo pero no sustituyen su revisión.

Lo que BugIt no hace

BugIt archiva en los once sistemas que nombra, cada uno con una credencial que usted crea en su propia cuenta y que BugIt comprueba contra el destino que elija antes de guardarla. Lo que llega a un campo de prioridad ordenable difiere según el sistema, y GitHub, GitLab y Trello no tienen ninguno, así que allí la gravedad se escribe en el texto del ticket. Confluence, Sentry y Notion son de solo lectura; todo lo demás se conecta mediante su propio servidor MCP compatible, que usted configura con ayuda de BugIt. Usa su IA (Copilot o su propia clave de OpenAI/Anthropic), no un modelo incluido; la censura de datos es en la medida de lo posible; y funciona en VS Code, en la extensión de Claude y en un terminal normal. Una licencia = un dispositivo (Solo) o hasta 5 miembros o puestos de dispositivo (Team). Los resultados y la consistencia dependen también de qué modelo de IA responda: uno más potente sigue los pasos de seguridad con más fiabilidad que uno muy ligero o gratuito.

PARTE 5 · SOLUCIÓN DE PROBLEMAS Y PREGUNTAS FRECUENTES

Cuando algo falle, descríbaselo primero al asistente en lenguaje sencillo. La mayoría de los problemas de configuración y de uso diario se pueden diagnosticar directamente en el editor.

✦ Pruebe esto primero: pídale a la IA que lo arregle

Cuando algo falle (durante la configuración o el uso diario), su solución más rápida es casi siempre **preguntarle al asistente directamente en el chat**. Describa lo que pasó con sus propias palabras (por ejemplo, "la configuración se detuvo a la mitad" o "mi sistema de seguimiento no se conecta") y pídale que lo arregle. La IA puede diagnosticar y reparar la mayoría de los problemas al instante. Por favor, pruebe esto antes de escribirnos; resuelve la gran mayoría de los problemas en segundos.

Aplicar una corrección de la IA: el botón "Keep"

Si el asistente corrige algo cambiando un archivo, VS Code muestra una pequeña barra Keep / Undo. Si pidió la corrección y está conforme con ella, haga clic en **Keep**, y el cambio se aplica solo a su propia copia, en su computadora. Haga clic en **Undo** para descartarlo. Nada de lo que mantenga con Keep se comparte con nadie más.

QUÉ VE

QUÉ HACER

"Activate to start"	Escriba <code>Activate</code> . Buglt abre el Buglt Portal en su navegador. Inicie sesión, elija su titularidad Solo o Team y apruebe este dispositivo. ¿Aún no tiene cuenta? Consiga una en la tienda.
El navegador no se abrió, o la activación no se completó	Escriba <code>Activate</code> de nuevo para reabrir el Buglt Portal, luego termine de iniciar sesión y de aprobar este dispositivo. La activación sucede por completo en el navegador; no hay ninguna clave que pegar.
"No sistema de seguimiento enabled"	Escriba <code>Begin setup</code> y elija su sistema de seguimiento.
Se rechaza el registro	Ejecute <code>python tools/connect.py jira --check</code> . Le dice quién es usted para el sistema de seguimiento, qué puede crear Buglt, y enumera en <code>fixes</code> el remedio concreto.
Sigue pidiéndole que active	Escriba <code>Activate</code> y complete el inicio de sesión en el navegador. Buglt nunca muestra los valores de las credenciales guardadas. (¿Primera vez? Consulte el Paso 8c.)
No registra un bug	Escriba <code>Check readiness</code> ; enumera exactamente lo que falta (normalmente que todavía no ha guardado el token de su sistema de seguimiento).
Algo parece roto después de una actualización	Escriba <code>Restore my settings</code> para retroceder.
El agente parece "salirse del guion"	Escriba <code>Read and follow all your given instructions</code> , o inicie un chat nuevo. Esto puede pasar con más frecuencia en un modelo de IA ligero/gratuito; un modelo más potente en el selector de modelos de Copilot es más consistente.
Las respuestas se sienten genéricas / no específicas del proyecto	Dele más contexto, o muéstrole un ticket existente durante <code>Begin setup</code> para que aprenda su estilo. Vuelva a ejecutar <code>Begin setup</code> para refinarlo.
Quiere cambiar una elección de la configuración	Escriba <code>Begin setup</code> y diga qué quiere cambiar (p. ej., "añade un sistema de seguimiento" o "empezar de nuevo") y reabra solo esa parte. Si la configuración simplemente se interrumpió a la mitad, un <code>Begin setup</code> simple continúa donde lo dejó en lugar de reiniciar.

Comprobaciones de estado integradas

`Check status` (¿están conectadas mis herramientas?) · `Check readiness` (¿qué falta antes de poder registrar?). Para una comprobación más profunda, abra Terminal → New Terminal y ejecute `python tools/selftest.py` .

Preguntas frecuentes

¿Necesito saber programar?

No. Si sabe instalar una app y escribir una frase, puede usar Buglt. Se encarga de las partes técnicas por usted.

¿Registrará bugs sin que yo lo sepa?

Nunca. Cada ticket y comentario se muestra en vista previa y necesita una confirmación escrita: `FILE IT` para comentar; un simple «sí» no basta. Use `dry run` para practicar sin ninguna escritura.

¿Tengo que iniciar un servidor cada vez?

No. El registro usa el token de API que guardó con `python tools/connect.py`, así que no hay nada que iniciar. Un servidor MCP solo hace falta para las integraciones opcionales de solo lectura, como Confluence, Sentry y Notion.

¿Puedo usarlo en dos computadoras?

Un dispositivo a la vez en Solo (hasta 5 miembros en Equipo, uno por miembro). Pasar a un dispositivo nuevo no es problema: active el nuevo, y el anterior deja de poder renovar de inmediato. Su plaza se libera en cuanto el dispositivo anterior se reconecta (y confirma que ha cedido el acceso) o su licencia sin conexión de 72 horas expira, así que no hay ningún periodo de espera fijo. Escriba `License status` para comprobarlo.

¿Qué pasa si el servidor de licencias está caído?

Sigue trabajando. Tras una activación en línea correcta, una licencia almacenada en caché le permite trabajar sin conexión hasta 72 horas, así que una interrupción del servidor de licencias (o simplemente un fin de semana sin internet) no lo detiene dentro de esa ventana. Pasado ese plazo se requiere una verificación en línea.

¿Necesito GitHub Copilot?

Es la forma más fácil. Si no lo tiene, ejecute `python tools/setup_wizard.py` en la Terminal para configurarlo, y Buglt puede ejecutarse de forma independiente con su propia clave de OpenAI/Anthropic (`python standalone/run.py`).

¿Son privados mis datos?

Sí. Taskivator solo recibe datos de licencia/actualización: el inicio de sesión en su cuenta de Buglt (en el navegador, en el Portal), un identificador de dispositivo con hash unidireccional y la titularidad Solo o Team que aprueba para este dispositivo. El software Buglt no utiliza la medición de Google Ads ni envía telemetría del producto. No hay ninguna clave de licencia, y su contraseña permanece en el navegador. Sus tickets solo van al modelo de IA y al sistema de seguimiento que conecte, nunca a nosotros.

¿Puedo editar los archivos de Buglt?

Está pensado para que personalice sus partes: su glosario, su estilo propio y sus ajustes (Parte 3). Solo no desactive la licencia/activación. Si encuentra un bug real en la herramienta misma, escriba a soporte para que se pueda corregir para todos en la próxima actualización.

¿Con qué sistemas de seguimiento funciona?

Con once, y Buglt archiva en todos ellos: Jira Cloud, Azure DevOps, GitHub Issues, GitLab Issues, Bugzilla, YouTrack, Linear, Shortcut, ClickUp, Asana y Trello. Cada uno tiene configuración guiada, cada uno escribe con una credencial que usted crea en su propia cuenta, y Buglt la comprueba contra su destino antes de guardarla. Lo que cambia es qué extras llegan: GitHub no tiene subida de adjuntos de ningún tipo, ClickUp acepta un archivo y no puede quitarlo, y GitHub, GitLab y Trello no tienen campo de prioridad, así que allí su gravedad se escribe en el texto del ticket. Elija el suyo durante la configuración y cámbielo cuando quiera con `Begin setup`.

¿Detectará bugs duplicados antes de que registre?

Sí. Antes de escribir nada, busca en su sistema de seguimiento reportes parecidos (incluso los redactados de forma un poco distinta) y le muestra las coincidencias, para que pueda evitar crear el mismo bug dos veces. Usted sigue decidiendo si continuar.

¿Puede escribir tickets en más de un idioma?

Sí. Añada un segundo idioma durante la configuración y cada reporte se escribe en ambos, en bloques separados, usando su glosario para que los términos del producto se mantengan exactos; nunca improvisa una traducción.

¿Puedo adjuntar una captura de pantalla?

Pida a BugIt que las adjunte. Crea una carpeta de evidencias, la abre y le indica la ruta exacta; deje ahí sus archivos y diga que ha terminado. BugIt elimina entonces cualquier cosa sensible que detecte, le muestra una vista previa y los adjunta tras su propio **FILE IT**. BugIt no puede acceder a un archivo pegado en el chat, por eso siempre le ofrece la carpeta.

¿Oculta contraseñas y tokens en mis reportes?

Cuando la redacción está activada, elimina correos, tokens y direcciones IP de cada borrador antes de registrarlo. Siempre ve el texto completo primero en su vista previa.

Registré un ticket por error. ¿Puedo deshacerlo?

La vista previa y la confirmación escrita (**FILE IT** ; un simple «sí» no basta) evitan la mayoría de los errores antes de escribir nada. Si activó el registro de auditoría, escriba **Undo last filing**. Si no, simplemente cierre o edite el ticket en su sistema de seguimiento como de costumbre.

¿Puede ayudarme a verificar o cerrar un bug?

Sí. Pida un comentario de verificación y redactará una nota clara de aprobado/fallido (en sus idiomas) sobre el ticket existente, y usted la aprueba antes de que se publique.

¿Puedo usarlo para más de un proyecto?

Cada carpeta de BugIt está configurada para el sistema de seguimiento de un proyecto. Para otro proyecto, o vuelve a ejecutar **Begin setup** para apuntarlo a otro sitio, o mantiene una copia separada por proyecto.

¿Cómo obtengo actualizaciones?

Cuando sale una versión nueva, BugIt la menciona al iniciar un chat. Escriba **update** y se instala in situ: su glosario, su estilo propio y sus ajustes se conservan, y antes hace una copia de seguridad.

¿Qué pasa cuando expira mi licencia?

Cuando termina su plazo, renueve desde su cuenta; su dispositivo aplica la titularidad renovada la próxima vez que se conecte en línea. El préstamo sin conexión de 72 horas mencionado arriba es independiente; solo cubre una interrupción temporal del servidor de licencias, no un plazo vencido. Escriba **License status** en cualquier momento.

Si renuevo o compro otra licencia, ¿se acumulan los días?

No, las licencias no se acumulan. Una renovación **reemplaza** su titularidad actual; su plazo empieza de cero y los días no usados no se trasladan, así que renueve cerca del final de su plazo.

Términos de la licencia (resumen en lenguaje sencillo)

Los términos completos y vinculantes están en el archivo [LICENSE](#) de su carpeta BugIt, y ese documento es el que rige. En resumen:

TEMA	EN PALABRAS SENCILLAS
Con licencia, no vendido	Compra una licencia para usar BugIt, no la propiedad. Taskivator conserva todos los derechos.
Sus puestos	Solo = un dispositivo a la vez; Equipo = hasta 5 miembros. Su cuenta y su titularidad son personales; no las comparta, revenda ni publique.
Revocación	Un acceso compartido, reembolsado, con contracargo o usado indebidamente puede ser revocado.
Personalice, pero...	Edita su config, glosario y plantillas libremente, pero no desactive ni evite la licencia/activación.
Su responsabilidad	Cómo usa BugIt y la seguridad de su proyecto son responsabilidad suya. Usted revisa y aprueba cada reporte antes de que se registre. Las salvaguardas son ayudas, no garantías.
"Tal cual", responsabilidad	Se proporciona tal cual, sin garantía. En la medida en que lo permita la ley, la responsabilidad se limita a lo que pagó, excluyendo daños indirectos o consecuentes.
Plazo y ley aplicable	Una licencia de un año que comienza el día que activa (una compra única, sin renovación automática), regida por las leyes de Japón. Los reembolsos se gestionan a través de la tienda donde compró.
Las renovaciones no se acumulan	Una renovación reemplaza su titularidad actual: su plazo empieza de cero en la activación y los días no usados no se trasladan, así que renueve cerca del final de su plazo. Cuando termina un plazo, renueve desde su cuenta y su dispositivo se vuelve a autorizar en su próxima comprobación en línea.

Dos documentos en su carpeta

[LICENSE](#) (términos completos) y [PRIVACY.md](#) (declaración de privacidad). Al activar y usar BugIt, acepta la LICENSE.

Cómo obtener ayuda

Por favor, revise primero la Solución de problemas y las Preguntas frecuentes de arriba; cubren casi todo. Luego, en este orden:

1 · Pídale a la IA que lo arregle

Su mejor primer movimiento: describa el problema en el chat y pídale al asistente que lo arregle. Si cambia un archivo y se ve bien, haga clic en Keep para aplicarlo (solo en su PC).

2 · Ejecute una comprobación de estado

[Check status](#) · [Check readiness](#) · o [python tools/selftest.py](#) en la Terminal.

3 · Restaure

[Restore my settings](#) deshace un cambio o una actualización problemática.

4 · Contacte a una persona

Solo si la guía y la IA no pudieron ayudarle: visite [bugit.dev](#) o escriba a support@bugit.dev (el soporte se atiende en inglés). ¿Problemas de configuración en sus primeros 7 días? Le pondremos en marcha o le ayudaremos a conseguir un reembolso a través de la tienda.

Por favor, mantenga la información confidencial de su proyecto fuera de los correos de soporte

Cada proyecto es diferente, y buena parte de su trabajo puede ser confidencial. Si nos escribe, por favor describa el problema de BugIt solo en términos generales, y no pegue código confidencial, detalles de bugs, especificaciones, capturas de pantalla ni datos de su proyecto. Taskivator no se hace responsable de ninguna información confidencial que nos envíe, y cualquier contenido confidencial que recibamos se elimina de inmediato. Siempre que sea posible, pídale ayuda primero al asistente de IA; puede resolver la mayoría de los problemas justo dentro de su editor.

Gracias por elegir BugIt.

Registre tickets limpios, evite los duplicados y recupere su tiempo, un bug a la vez.

BugIt QA Agent · Guía del usuario · © 2026 Taskivator. All Rights Reserved. · [bugit.dev](#) · Los archivos LICENSE y PRIVACY.md incluidos con su descarga son los documentos vinculantes.